

CC3001 Algoritmos y Estructuras de Datos**Profesores:** Nelson Baloian, Patricio Poblete, e Iván Sipirán**Auxiliares:** Sebastián Acuña, Daniela Espinoza, Cristián Llull Torres,

Benjamín Osses, Anish Samtani, y Marcelo Zamorano

**Auxiliar 7: Splay Trees, Skip Lists e Intro Hashing**

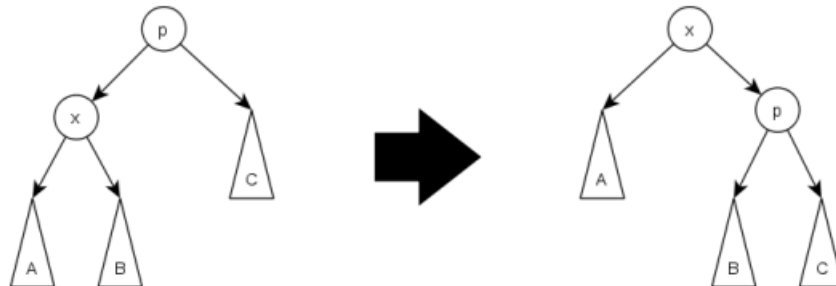
05 de junio de 2026

P1. Splay Trees

Un splay tree (o árbol biselado) es otro tipo de árbol de búsqueda binaria auto-balanceante, cuya propiedad más interesante es que los últimos nodos buscados e insertados están más cerca de la raíz que los nodos más antiguos. En particular, el último nodo insertado/accedido x va a ser la nueva raíz del árbol. Esto se logra mediante una operación llamada splaying en x .

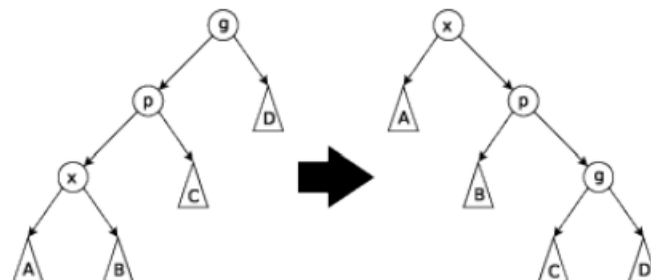
Esta estrategia garantiza que cualquier secuencia de m operaciones en un árbol que llega a tener n elementos, partiendo de un árbol vacío, toma tiempo $O(m \log n)$. Es importante notar que no garantiza que alguna secuencia en particular no tome $O(n)$, sino que el costo acumulado, dividido por el número de operaciones, da un promedio de $O(\log n)$ por operación. Se dice que una estructura de este tipo es eficiente en el sentido amortizado. Hay tres casos para ejecutar operación de splaying:

- **Caso 1: zig**



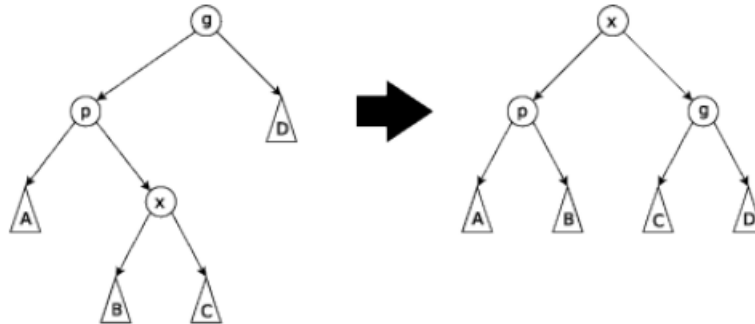
El padre de x es la raíz. Se hace una rotación simple sobre el padre (al igual que en árboles AVL), dejando a x como la nueva raíz.

- **Caso 2: zig-zig**



Tanto x como su padre son hijos del mismo lado (izquierdo o derecho) de su abuelo. El zig-zig se convierte en zag-zag, resultando en un árbol con x en la raíz, el padre de x como su hijo, y el abuelo de x como su nieto.

- **Caso 3: zig-zag**

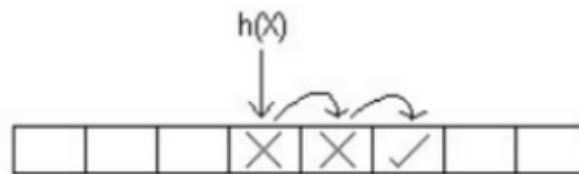


El nodo x es hijo del lado derecho de su padre, y el padre es hijo izquierdo del abuelo (están en zigzag). El caso zag-zig es análogo. Se hace una rotación simple entre x y su padre. Luego, se rota x con su abuelo (ahora el padre) como en el caso 1 (en la imagen, el árbol $(p \ A \ B)$ cumple el rol del árbol A resultante en el diagrama del caso 1). Esto es igual a la rotación doble de los árboles AVL.

Si x no se encuentra en el árbol, se realiza la operación $\text{splay}(x')$, con x' el último nodo visitado.

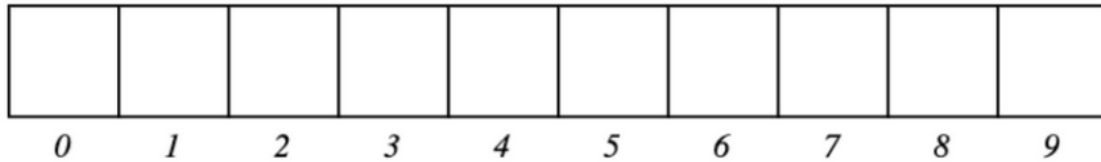
- Inserte en un Splay Tree vacío la siguiente secuencia: 1, 2, 3, 4, 5, 6, 7, 8.
- Acceda al nodo con valor 1 y realice el splaying correspondiente. Comente acerca de la complejidad de búsqueda en este caso en particular.
- Practique accediendo a los nodos con valores 4 y 5. Termine insertando el 9.

P2. Linear Probing



Suponga que se tiene una tabla de hashing con Linear Probing, de tamaño 10, inicialmente vacía, con la función de hashing $h(x) = x \bmod 10$ (por ejemplo, $h(64) = 4$).

Muestre en la siguiente tabla el resultado de insertar (a mano) la siguiente secuencia de llaves: 34, 59, 45, 27, 14, 22, 75, 25



P3. Skip Lists

Una skip list es una lista enlazada que contiene nodos de alturas distintas. El nodo cabeza y el último nodo son de altura máxima, y de cada nodo de altura i surgen i referencias hacia nodos más adelante en la lista, donde la referencia a la altura j lleva al nodo de altura $k \geq j$ más cercano.

Con esta estructura, asignando el tamaño de los nodos al azar, se pueden obtener búsquedas tan eficientes como la búsqueda binaria (¿qué costo tiene la búsqueda binaria en un arreglo ordenado?). Se sugiere estudiar la sección de *skip lists* en los apuntes.

Dada una clase `SkipList`, se le pide:

- (a) Completar la implementación de la operación `buscar` en una skip list.
- (b) Implementar la operación `insertar` en una skip list.

Propuesto:

- Generar un programa que inserte los números del 1 al 500 en la skip list, y generar un arreglo de Numpy con los números enteros del 1 al 500, para finalmente comparar el costo de tiempo de búsqueda entre la skip list, la búsqueda binaria y la búsqueda secuencial. Asuma que la altura máxima de los nodos en la skip list es de $\frac{n}{2}$, con n la cantidad de elementos a insertar:
 - (a) Buscar los números del 0 al 9 (10 números).
 - (b) Buscar los números del 0 al 99 (100 números).
 - (c) Buscar los números del 0 al 999 (1000 números).

```
max_int = 2**31 - 1
min_int = -(2**31)
```

```
class Pila:
    def __init__(self):
        self.s = []

    def push(self, x):
        self.s.append(x)

    def pop(self):
        assert len(self.s) > 0
        return self.s.pop() # pop de lista, no de Pila

    def is_empty(self):
        return len(self.s) == 0
```

```
class Nodo:
    def __init__(self, val, alt_max, rand_alt=True):
        self.val = val
        # La altura es equivalente a la cantidad de caras obtenidas
        # al lanzar una moneda alt_max veces menos 1 (entre 0 y alt_max -1),
        # más 1 (entre 1 y alt_max)
        if rand_alt:
            altura = (np.random.binomial(alt_max - 1, 0.5, 1) + 1)[0]
        else:
            altura = alt_max
        self.nodos = np.zeros(altura, dtype="object") # punteros con null

class SkipList:
    def __init__(self, alt_max=1):
        self.alt_max = alt_max
        self.cabeza = Nodo(min_int, self.alt_max, rand_alt=False)
        self.ultimo = Nodo(max_int, self.alt_max, rand_alt=False)
        for i in range(alt_max):
            self.cabeza.nodos[i] = self.ultimo

    def insertar(self, x):
        pass

    def buscar(self, x):
        pass

    def imprimir(self):
        pass
```